

Date: 30/10/2025	Write java programs that implement the following A) Class and object B) Constructor C) Parameterized Constructor D) Method Overloading E) Constructor Overloading
Experiment:01	

A) Class and object:
Source code:

```
class Student {  
    String name;  
    int age;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
public class ClassObjectDemo {  
    public static void main(String[] args) {  
        Student s = new Student(); // Creating object  
        s.name = "Rakesh";  
        s.age = 22;  
        s.display();  
    }  
}
```

OUTPUT:
Name: Rakesh
Age: 22

B) Constructor:**Source code:**

```
class Demo {  
    Demo() { // Default constructor  
        System.out.println("Constructor is called!");  
    }  
}  
  
public class ConstructorDemo {  
    public static void main(String[] args) {  
        Demo d = new Demo(); // Calls constructor automatically  
    }  
}
```

OUTPUT:

Constructor is called!

C) Parameterized Constructor:**Source code:**

```
class Employee {  
    String name;  
    int id;  
  
    Employee(String n, int i) { // Parameterized constructor  
        name = n;  
        id = i;  
    }  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("ID: " + id);  
    }  
}  
  
public class ParameterizedDemo {  
    public static void main(String[] args) {  
        Employee e = new Employee("Rakesh", 101);  
        e.display();  
    }  
}
```

OUTPUT:

```
Name: Rakesh  
ID: 101
```

D) Method Overloading:**Source code:**

```
class MathOperation {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    int add(int a, int b, int c) { // Overloaded method  
        return a + b + c;  
    }  
}  
  
public class OverloadDemo {  
    public static void main(String[] args) {  
        MathOperation m = new MathOperation();  
        System.out.println("Sum (2 numbers): " + m.add(10, 20));  
        System.out.println("Sum (3 numbers): " + m.add(10, 20, 30));  
    }  
}
```

OUTPUT:

```
Sum (2 numbers): 30  
Sum (3 numbers): 60
```

E) Constructor Overloading:**Source code:**

```
class Person {  
    String name;  
    int age;  
  
    Person() {           // Default  
        name = "Unknown";  
        age = 0;  
    }  
  
    Person(String n) {    // One parameter  
        name = n;  
        age = 18;  
    }  
  
    Person(String n, int a) { // Two parameters  
        name = n;  
        age = a;  
    }  
    void display() {  
        System.out.println("Name: " + name + ", Age: " + age);  
    }  
}  
  
public class ConstructorOverloadDemo {  
    public static void main(String[] args) {  
        Person p1 = new Person();  
        Person p2 = new Person("Rakesh");  
        Person p3 = new Person("Rakesh", 22);  
  
        p1.display();  
        p2.display();  
        p3.display();  
    }  
}
```

OUTPUT:

Name: Unknown, Age: 0

Name: Rakesh, Age: 18

Name: Rakesh, Age: 22

Date: 30/10/2025**Experiment:2A)****Write a Java program that checks whether a given string is a palindrome or not. Ex: MADAM is a palindrome.****Source code:**

```
import java.util.Scanner;

public class PalindromeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        // Convert to uppercase for case-insensitive comparison
        String original = str.toUpperCase();
        String reverse = "";

        // Reverse the string
        for (int i = original.length() - 1; i >= 0; i--) {
            reverse += original.charAt(i);
        }

        // Check palindrome
        if (original.equals(reverse)) {
            System.out.println(str + " is a palindrome.");
        } else {
            System.out.println(str + " is not a palindrome.");
        }

        sc.close();
    }
}
```

OUTPUT:

```
Enter a string: MADAM
MADAM is a palindrome.
```

Date: 30/10/2025**Experiment:2B)****Write a Java program for sorting a given list of names in ascending order.****Source code:**

```
import java.util.*;

public class SortNames {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of names: ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        String[] names = new String[n];

        // Reading names
        System.out.println("Enter the names:");
        for (int i = 0; i < n; i++) {
            names[i] = sc.nextLine();
        }

        // Sorting names
        Arrays.sort(names);

        // Displaying sorted names
        System.out.println("\nNames in Ascending Order:");
        for (String name : names) {
            System.out.println(name);
        }

        sc.close();
    }
}
```

OUTPUT:

```
Enter number of names: 3
Enter the names:
Vishal
rakesh

Names in Ascending
Order:
Vishal
```

Date: 30/10/2025**Experiment:2C)****Write a Java Program that reads a line of integers, and then displays each integer and the sum of all the integers (Use String Tokenizer class of java.util)****Source code:**

```
import java.util.Scanner;
import java.util.StringTokenizer;

public class TokenizerExample {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.println("Enter a line of integers:");
        String input = sc.nextLine();

        StringTokenizer st = new StringTokenizer(input);

        int sum = 0;

        System.out.println("\nIntegers are:");
        while (st.hasMoreTokens()) {
            String token = st.nextToken();
            int num = Integer.parseInt(token);
            System.out.println(num);
            sum += num;
        }

        System.out.println("Sum of integers: " + sum);

        sc.close();
    }
}
```

OUTPUT:**Enter a line of integers:****2 5 6****Integers are:****2****5****6****Sum of integers: 13**

Date: 06/11/2025	Write java programs that uses the following keywords a) this b)super c)static d)final
Experiment:3)	

A) This keyword:
Source code:

```
class Student {  
    int id;  
    String name;  
  
    Student(int id, String name) {  
        this.id = id;    // using 'this' to refer to instance variable  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("ID: " + this.id);  
        System.out.println("Name: " + this.name);  
    }  
}  
  
public class ThisDemo {  
    public static void main(String[] args) {  
        Student s = new Student(101, "Rakesh");  
        s.display();  
    }  
}
```

OUTPUT:
ID: 101
Name: Rakesh

B) Super keyword:**Source code:**

```
class Animal {  
    String sound = "Generic Sound";  
  
    void makeSound() {  
        System.out.println("Animal makes: " + sound);  
    }  
}  
  
class Dog extends Animal {  
    String sound = "Bark";  
  
    void show() {  
        System.out.println("Dog makes: " + sound);  
        System.out.println("Parent sound using super: " + super.sound);  
  
        super.makeSound(); // calling parent method  
    }  
}  
  
public class SuperDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.show();  
    }  
}
```

OUTPUT:

```
Dog makes: Bark  
Parent sound using super: Generic Sound  
Animal makes: Generic Sound
```

C) Static keyword:**Source code:**

```
static int count = 0;                                // static variable

static {                                              // static block
    System.out.println("Static block executed");
}

StaticDemo() {
    count++;                                          // increments shared static variable
}

static void showCount() {                            // static method
    System.out.println("Object count = " + count);
}

}

public class StaticTest {
    public static void main(String[] args) {

        StaticDemo s1 = new StaticDemo();
        StaticDemo s2 = new StaticDemo();
        StaticDemo s3 = new StaticDemo();

        StaticDemo.showCount();    // calling static method
    }
}
```

OUTPUT:

```
Static block executed
Object count = 3
```

D) Final keyword:**Source code:**

```
final class Vehicle {                                // final class
    final int speed = 80;                             // final variable

    final void display() {                            // final method
        System.out.println("Speed: " + speed);
    }
}

public class FinalDemo {
    public static void main(String[] args) {

        Vehicle v = new Vehicle();
        v.display();

        // v.speed = 100; // ERROR: cannot change final variable
    }
}
```

OUTPUT:

Speed: 80

Date: 06/11/2025	Write a java program to implement method overriding.
Experiment:4A)	

Source code:

```
class Animal {  
    void sound() {  
        System.out.println("Animals make some sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class MethodOverridingDemo {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound(); // Calls overridden method  
    }  
}
```

OUTPUT:

Dog barks

Date: 06/11/2025**Experiment:4B)****Write a java program to implement dynamic method dispatch.****Source code:**

```
class Parent {  
    void show() {  
        System.out.println("Parent's show method");  
    }  
}  
  
class Child1 extends Parent {  
    void show() {  
        System.out.println("Child1's show method");  
    }  
}  
  
class Child2 extends Parent {  
    void show() {  
        System.out.println("Child2's show method");  
    }  
}  
  
public class DynamicDispatchDemo {  
    public static void main(String[] args) {  
        Parent ref;  
  
        ref = new Child1();  
        ref.show(); // Runtime: Child1's method executes  
  
        ref = new Child2();  
        ref.show(); // Runtime: Child2's method executes  
    }  
}
```

OUTPUT :

Child1's show method
Child2's show method

Date: 06/11/2025**Experiment:4C)****Write a Java program to implement multiple inheritance.****Source code:**

```
interface A {  
    void displayA();  
}  
  
interface B {  
    void displayB();  
}  
  
class C implements A, B {  
    public void displayA() {  
        System.out.println("Inside A interface method");  
    }  
  
    public void displayB() {  
        System.out.println("Inside B interface method");  
    }  
}  
  
public class MultipleInheritanceDemo {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.displayA();  
        obj.displayB();  
    }  
}
```

OUTPUT:

```
Inside A interface method  
Inside B interface method
```

Date: 06/11/2025

Experiment:4D)

Write a java program that uses access specifiers.

Source code:

```
class Person {
    public String name = "Rakesh";    // accessible everywhere
    private int age = 22;              // accessible only inside class
    protected String city = "Hyderabad"; // accessible in same package + subclasses
    String country = "India";          // default access (package-private)

    public void showDetails() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);    // allowed
        System.out.println("City: " + city);
        System.out.println("Country: " + country);
    }
}

public class AccessSpecifiersDemo {
    public static void main(String[] args) {
        Person p = new Person();
        p.showDetails();

        System.out.println("\nAccessing fields directly:");
        System.out.println(p.name);    // public → accessible

        // System.out.println(p.age);    // private → ERROR
        System.out.println(p.city);    // protected → accessible (same package)
        System.out.println(p.country); // default → accessible (same package)
    }
}
```

OUTPUT:

```
Name: Rakesh
Age: 22
City: Hyderabad
Country: India
Accessing fields directly:
Rakesh
Hyderabad
India
```


Date: 13/11/2025	Write a Java program that reads a file name from the user, then displays information about whether the file exists, whether the file is readable, whether the file is writable, the type of file and the length of the file in bytes.
Experiment:5A)	

Source code:

```
import java.io.*;
import java.util.Scanner;

public class FileInfo {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter file name: ");
        String fname = sc.nextLine();

        File file = new File(fname);

        if (!file.exists()) {
            System.out.println("File does not exist.");
        } else {
            System.out.println("File exists: Yes");
            System.out.println("Readable: " + file.canRead());
            System.out.println("Writable: " + file.canWrite());

            if (file.isFile())
                System.out.println("Type: Regular File");
            else if (file.isDirectory())
                System.out.println("Type: Directory");
            else
                System.out.println("Type: Unknown");

            System.out.println("File Size: " + file.length() + " bytes");
        }
        sc.close();
    }
}
```

```
}  
}
```

OUTPUT:

Enter file name: hi.txt
File exists: Yes
Readable: true
Writable: true
Type: Regular File
File Size: 9 bytes

Date: 13/11/2025**Experiment:5B)****Write a Java program that reads a file and displays the file on the screen, with a line number before each line.****Source code:**

```
import java.io.*;
import java.util.*;
public class DisplayFile {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String fname = sc.nextLine();

        try {
            BufferedReader br = new BufferedReader(new FileReader(fname));

            String line;
            int lineNumber = 1;
            while ((line = br.readLine()) != null) {
                System.out.println(lineNumber + ": " + line);
                lineNumber++;
            }
            br.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("File not found.");
        }
        catch (IOException e) {
            System.out.println("Error reading file.");
        }
        sc.close();
    }
}
```

OUTPUT:

Enter file name: hi.txt
1: dfgdsfgsd

Date: 13/11/2025**Experiment:5C)****Write a Java program that displays the number of characters, lines and words in a text file.****Source code:**

```
import java.io.*;
import java.util.Scanner;

public class FileCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter file name: ");
        String fileName = sc.nextLine();

        int lines = 0, words = 0, chars = 0;

        try {
            BufferedReader br = new BufferedReader(new FileReader(fileName));
            String line;

            while ((line = br.readLine()) != null) {
                lines++;           // count lines
                chars += line.length(); // count characters

                // split into words
                String[] w = line.trim().split("\\s+");
                if (!line.trim().isEmpty()) {
                    words += w.length;
                }
            }

            br.close();

            System.out.println("Number of lines: " + lines);
            System.out.println("Number of words: " + words);
        }
    }
}
```

```
        System.out.println("Number of characters: " + chars);
    }
    catch (FileNotFoundException e) {
        System.out.println("File not found.");
    }
    catch (IOException e) {
        System.out.println("Error reading file.");
    }
    }

    sc.close();
}
}
```

OUTPUT:

Enter file name: hi.txt
Number of lines: 1
Number of words: 1
Number of characters: 9

Date: 13/11/2025**Experiment:6A)****Write a Java program for handling Checked Exceptions.****Source code:**

```
import java.io.*;

public class CheckedExceptionExample {
    public static void main(String[] args) {

        try {
            FileReader fr = new FileReader("hi.txt"); // Checked Exception
            BufferedReader br = new BufferedReader(fr);

            String line = br.readLine();           // May throw IOException
            System.out.println("First line: " + line);

            br.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("Error: The file does not exist.");
        }
        catch (IOException e) {
            System.out.println("Error: Problem occurred while reading the file.");
        }

        System.out.println("Program continues normally...");
    }
}
```

OUTPUT:

```
First line: dfgdsfgsd
Program continues normally...
```

Date: 13/11/2025**Experiment:6B)****Write a Java program for handling Unchecked Exceptions.****Source code:**

```
public class UncheckedExceptionExample {  
    public static void main(String[] args) {  
  
        try {  
            int a = 10;  
            int b = 0;  
  
            int result = a / b; // ArithmeticException  
  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[5]); // ArrayIndexOutOfBoundsException  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Error: Division by zero is not allowed.");  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Error: Array index is out of range.");  
        }  
  
        System.out.println("Program continues normally...");  
    }  
}
```

OUTPUT:

Error: Division by zero is not allowed.
Program continues normally...

Date: 20/11/2025	Write a Java program that creates three threads. First thread displays “Good Morning” every one second, the second thread displays “Hello” every two seconds and the third thread displays “Welcome” every three seconds.
Experiment:7A)	

Source code:

```
class MessageThread extends Thread {
    String message;
    int delay;

    MessageThread(String message, int delay) {
        this.message = message;
        this.delay = delay;
    }

    public void run() {
        try {
            while (true) {
                System.out.println(message);
                Thread.sleep(delay);
            }
        } catch (InterruptedException e) {
            System.out.println("Thread interrupted");
        }
    }
}

public class ThreeThreads {
    public static void main(String[] args) {

        MessageThread t1 = new MessageThread("Good Morning", 1000);
        MessageThread t2 = new MessageThread("Hello", 2000);
        MessageThread t3 = new MessageThread("Welcome", 3000);

        t1.start();
        t2.start();
        t3.start();
    }
}
```



```
}  
}
```

OUTPUT:

Good Morning
Welcome
Hello
Good Morning
Hello
Good Morning
Good Morning
Welcome
Hello
Good Morning
Good Morning
Hello
Welcome
Good Morning
Good Morning

Date: 20/11/2025	Write a Java program that correctly implements producer consumer problem using the concept of inter thread communication.
Experiment:7B)	

Source code:

```
class Buffer {
    int value;
    boolean ready = false;

    synchronized void produce(int v) throws InterruptedException {
        while (ready) wait();
        value = v;
        ready = true;
        System.out.println("Produced: " + v);
        notify();
    }

    synchronized int consume() throws InterruptedException {
        while (!ready) wait();
        ready = false;
        System.out.println("Consumed: " + value);
        notify();
        return value;
    }
}

class Producer extends Thread {
    Buffer b;
    Producer(Buffer b) { this.b = b; }

    public void run() {
        int i = 1;
        try {
            while (true) b.produce(i++);
        } catch (Exception e) {}
    }
}
```

```
}  
  
class Consumer extends Thread {  
    Buffer b;  
    Consumer(Buffer b) { this.b = b; }  
  
    public void run() {  
        try {  
            while (true) b.consume();  
        } catch (Exception e) {}  
    }  
}  
  
public class ProducerConsumer {  
    public static void main(String[] args) {  
        Buffer b = new Buffer();  
        new Producer(b).start();  
        new Consumer(b).start();  
    }  
}
```

OUTPUT:

```
Consumed: 8504  
Produced: 8505  
Consumed: 8505  
Produced: 8506  
Consumed: 8506  
Produced: 8507  
Consumed: 8507  
Produced: 8508  
Consumed: 8508
```

Date: 04/12/2025**Experiment:8)****Write a program illustrating following collections framework**
a) Array List b) Vector c) Hash Table d) Stack**A) Array List:****Source code:**

```
import java.util.ArrayList;

public class ArrayListDemo {
    public static void main(String[] args) {
        // Creating ArrayList
        ArrayList<String> fruits = new ArrayList<>();
        // Adding elements
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Cherry");
        fruits.add("Mango");
        // Accessing elements
        System.out.println("ArrayList Elements: " + fruits);
        // Removing an element
        fruits.remove("Banana");
        System.out.println("After removing Banana: " + fruits);
        // Checking size
        System.out.println("Size of ArrayList: " + fruits.size());
    }
}
```

OUTPUT:

```
ArrayList Elements: [Apple, Banana, Cherry, Mango]
After removing Banana: [Apple, Cherry, Mango]
Size of ArrayList: 3
```

B) Vector:**Source code:**

```
import java.util.Vector;

public class VectorDemo {
    public static void main(String[] args) {

        // Creating Vector
        Vector<Integer> numbers = new Vector<>();

        // Adding elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);
        numbers.add(40);

        System.out.println("Vector elements: " + numbers);

        // Checking capacity and size
        System.out.println("Size: " + numbers.size());
        System.out.println("Capacity: " + numbers.capacity());

        // Removing element
        numbers.remove(2);
        System.out.println("After removing index 2: " + numbers);
    }
}
```

OUTPUT:

```
Size: 4
Capacity: 10
After removing index 2: [10, 20, 40]
```

C) HashTable :**Source code:**

```
import java.util.Hashtable;

public class Hash {
    public static void main(String[] args) {

        Hashtable<Integer, String> table = new Hashtable<>();

        // Adding key-value pairs
        table.put(1, "One");
        table.put(2, "Two");
        table.put(3, "Three");

        System.out.println("Hashtable: " + table);

        // Accessing a value
        System.out.println("Value for key 2: " + table.get(2));

        // Removing a key-value pair
        table.remove(3);
        System.out.println("After removing key 3: " + table);
    }
}
```

OUTPUT:

```
Hashtable: {3=Three, 2=Two, 1=One}
Value for key 2: Two
After removing key 3: {2=Two, 1=One}
```

D) Stack:**Source code:**

```
import java.util.Stack;

public class StackDemo {
    public static void main(String[] args) {

        Stack<String> colors = new Stack<>();

        // Pushing elements
        colors.push("Red");
        colors.push("Green");
        colors.push("Blue");
        colors.push("Yellow");

        System.out.println("Stack elements: " + colors);

        // Peek top of stack
        System.out.println("Top element: " + colors.peek());

        // Pop an element
        String popped = colors.pop();
        System.out.println("Popped element: " + popped);

        System.out.println("Stack after pop: " + colors);
    }
}
```

OUTPUT:

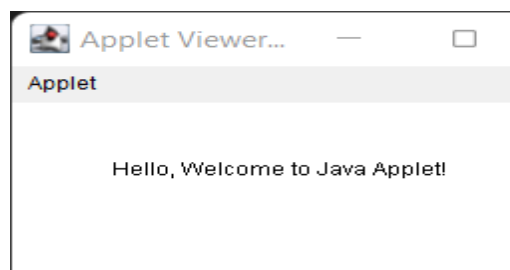
```
Stack elements: [Red, Green, Blue, Yellow]
Top element: Yellow
Popped element: Yellow
Stack after pop: [Red, Green, Blue]
```

Date: 18/12/2025**Experiment:9A)****Develop an applet that displays a simple message.****Source code:**

```
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class SimpleMessageApplet extends Applet {  
  
    public void paint(Graphics g) {  
        g.drawString("Hello, Welcome to Java Applet!", 50, 50);  
    }  
}
```

SimpleMessageApplet.html

```
<html>  
<body>  
<applet code="SimpleMessageApplet.class" width="300" height="100">  
</applet>  
</body>  
</html>
```

OUTPUT:

Date: 18/12/2025	Develop an applet that receives an integer in one text field and compute its factorial value and return it in another text field, when the button named “Compute” is clicked.
Experiment:9B)	

Source code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class FactorialApplet extends Applet implements ActionListener {

    Label l1, l2;
    TextField t1, t2;
    Button b;

    public void init() {
        l1 = new Label("Enter Number:");
        t1 = new TextField(10);

        l2 = new Label("Factorial:");
        t2 = new TextField(10);
        t2.setEditable(false);

        b = new Button("Compute");
        b.addActionListener(this);

        add(l1);
        add(t1);
        add(l2);
        add(t2);
        add(b);
    }

    public void actionPerformed(ActionEvent e) {
        try {
            int n = Integer.parseInt(t1.getText());
```

```
        long fact = 1;

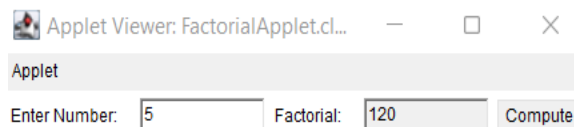
        for (int i = 1; i <= n; i++) {
            fact *= i;
        }

        t2.setText(String.valueOf(fact));
    } catch (Exception ex) {
        t2.setText("Invalid Input");
    }
}
}
```

FactorialApplet.html

```
<html>
<body>
<applet code="FactorialApplet.class" width="300" height="200">
</applet>
</body>
</html>
```

OUTPUT:



Applet started.

Date: 18/12/2025	Write a Java program that works as a simple calculator. Use a grid layout to arrange button for the digits and for the +,-,*, % operations. Add a text field to display the result.
Experiment:9C)	

Source code:

```
import java.awt.*;
import java.awt.event.*;

public class SimpleCalc extends Frame implements ActionListener {

    TextField tf;
    double num1, num2, result;
    char operator;

    public SimpleCalc() {
        tf = new TextField(20);
        add(tf, BorderLayout.NORTH);

        Panel p = new Panel();
        p.setLayout(new GridLayout(4, 4));

        String buttons[] = { "7","8","9","+",
                             "4","5","6","-",
                             "1","2","3","*",
                             "0","C","=","/" };

        for (String b : buttons) {
            Button btn = new Button(b);
            btn.addActionListener(this);
            p.add(btn);
        }

        add(p);

        setSize(250, 300);
        setVisible(true);
    }
}
```

```
addWindowListener(new WindowAdapter(){
    public void windowClosing(WindowEvent e){
        System.exit(0);
    }
});

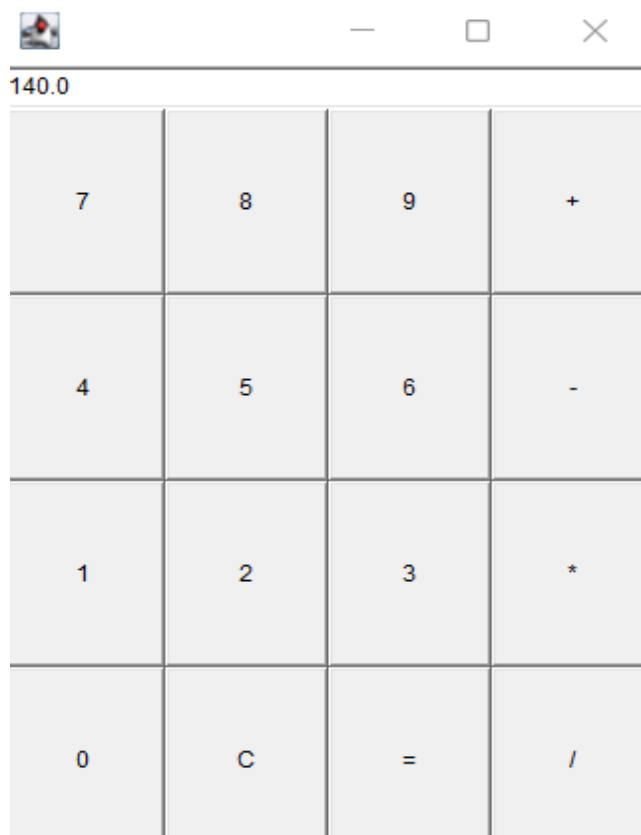
public void actionPerformed(ActionEvent e) {
    String s = e.getActionCommand();

    if (s.charAt(0) >= '0' && s.charAt(0) <= '9') {
        tf.setText(tf.getText() + s);
    }
    else if (s.equals("C")) {
        tf.setText("");
    }
    else if (s.equals("=")) {
        num2 = Double.parseDouble(tf.getText());

        switch(operator) {
            case '+': result = num1 + num2; break;
            case '-': result = num1 - num2; break;
            case '*': result = num1 * num2; break;
            case '/': result = num1 / num2; break;
        }

        tf.setText("" + result);
    }
    else {
        num1 = Double.parseDouble(tf.getText());
        operator = s.charAt(0);
        tf.setText("");
    }
}

public static void main(String[] args) {
    new SimpleCalc();
}
```

OUTPUT:

Date: 07/01/2026**Experiment:10A)****Write a Java program for handling mouse events.****Source code:**

```
import java.awt.*;
import java.awt.event.*;

public class MouseEventsDemo extends Frame implements MouseListener {

    String msg = "";

    public MouseEventsDemo() {
        addMouseListener(this);

        setSize(400, 300);
        setTitle("Mouse Events Demo");
        setVisible(true);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    public void paint(Graphics g) {
        g.drawString(msg, 20, 100);
    }

    public void mouseClicked(MouseEvent e) {
        msg = "Mouse Clicked";
        repaint();
    }

    public void mouseEntered(MouseEvent e) {
```

```
        msg = "Mouse Entered";  
        repaint();  
    }  
  
    public void mouseExited(MouseEvent e) {  
        msg = "Mouse Exited";  
        repaint();  
    }  
  
    public void mousePressed(MouseEvent e) {  
        msg = "Mouse Pressed";  
        repaint();  
    }  
  
    public void mouseReleased(MouseEvent e) {  
        msg = "Mouse Released";  
        repaint();  
    }  
  
    public static void main(String[] args) {  
        new MouseEventsDemo();  
    }  
}
```

OUTPUT: Mouse Events Demo

- □ X

Mouse Exited

Date: 07/01/2026**Experiment:10B)****Write a Java program for handling key events.****Source code:**

```
import java.awt.*;
import java.awt.event.*;

public class KeyEvents extends Frame implements KeyListener {

    String msg = "";

    public KeyEvents() {
        addKeyListener(this);

        setSize(400, 300);
        setTitle("Key Events Demo");
        setVisible(true);

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    public void paint(Graphics g) {
        g.drawString("Pressed Key: " + msg, 50, 150);
    }

    public void keyPressed(KeyEvent e) {
        msg = "Key Pressed: " + e.getKeyChar();
        repaint();
    }

    public void keyReleased(KeyEvent e) {
```



```
        msg = "Key Released: " + e.getKeyChar();
        repaint();
    }

    public void keyTyped(KeyEvent e) {
        msg = "Key Typed: " + e.getKeyChar();
        repaint();
    }

    public static void main(String[] args) {
        new KeyEvents();
    }
}
```

OUTPUT:

Pressed Key: Key Released: □

Date: 08/01/2026	Write a program that creates a user interface to perform integer divisions. The user enters two numbers in the text fields Num1 and Num 2.
Experiment:11A)	

Source code:

```
import java.awt.*;
import java.awt.event.*;

public class IntegerDivision extends Frame implements ActionListener {

    Label l1, l2, l3;
    TextField t1, t2, t3;
    Button divide;

    public IntegerDivision() {

        l1 = new Label("Num 1:");
        l2 = new Label("Num 2:");
        l3 = new Label("Result:");

        t1 = new TextField(10);
        t2 = new TextField(10);
        t3 = new TextField(10);
        t3.setEditable(false);

        divide = new Button("Divide");
        divide.addActionListener(this);

        setLayout(new FlowLayout());

        add(l1); add(t1);
        add(l2); add(t2);
        add(divide);
        add(l3); add(t3);

        setSize(300, 200);
```

```
setTitle("Integer Division");
setVisible(true);

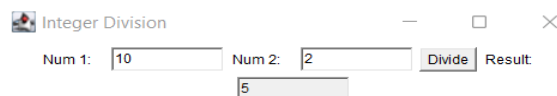
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});

public void actionPerformed(ActionEvent e) {
    try {
        int a = Integer.parseInt(t1.getText());
        int b = Integer.parseInt(t2.getText());

        if (b == 0) {
            t3.setText("Cannot divide");
        } else {
            int result = a / b;
            t3.setText(String.valueOf(result));
        }

    } catch (Exception ex) {
        t3.setText("Invalid input");
    }
}

public static void main(String[] args) {
    new IntegerDivision();
}
```

OUTPUT:

Date: 08/01/2026	The division of Num1 and Num2 is displayed in the Result field when the Divide button is clicked. If Num1 or Num2 were not an integer, the program would throw Number Format Exception. If Num2 were Zero, the program would throw an Arithmetic Exception and display the exception in a message dialog box.
Experiment:11B)	

Source code:

```
import java.awt.*;
import java.awt.event.*;

public class IntegerDivisionException extends Frame implements
ActionListener {

    TextField t1, t2, result;
    Button divideBtn;

    public IntegerDivisionException() {

        setLayout(new FlowLayout());

        add(new Label("Num1:"));
        t1 = new TextField(10);
        add(t1);

        add(new Label("Num2:"));
        t2 = new TextField(10);
        add(t2);

        divideBtn = new Button("Divide");
        add(divideBtn);

        add(new Label("Result:"));
        result = new TextField(10);
        result.setEditable(false);
        add(result);

        divideBtn.addActionListener(this);
```

```
setSize(300, 200);
setTitle("Integer Division");
setVisible(true);

addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
});
}

public void actionPerformed(ActionEvent e) {

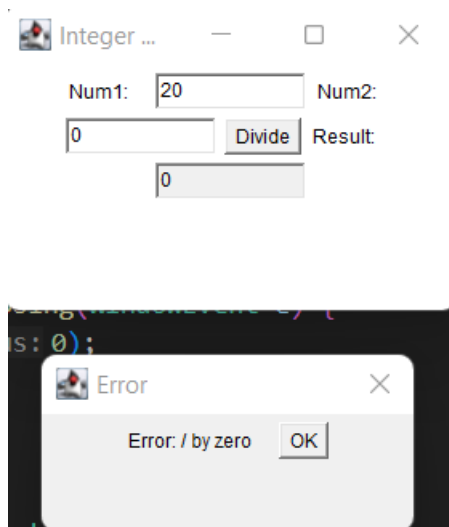
    try {
        int num1 = Integer.parseInt(t1.getText());
        int num2 = Integer.parseInt(t2.getText());

        // Arithmetic Exception (division by zero)
        int res = num1 / num2;

        result.setText(String.valueOf(res));
    }
    catch (NumberFormatException nfe) {
        showMessage("Invalid Number! Please enter integers only.");
    }
    catch (ArithmeticException ae) {
        showMessage("Error: " + ae.getMessage());
    }
}

// Simple dialog box to show errors
void showMessage(String msg) {
    Dialog d = new Dialog(this, "Error", true);
    d.setLayout(new FlowLayout());
    d.add(new Label(msg));
    Button ok = new Button("OK");
    ok.addActionListener(e -> d.setVisible(false));
    d.add(ok);
    d.setSize(250, 120);
    d.setVisible(true);
}
```

```
}  
  
public static void main(String[] args) {  
    new IntegerDivisionException();  
}  
}
```

OUTPUT:

Date: 22/01/2026	Write a java program that simulates traffic light. The program lets the user select one of three lights: red, yellow, or green. When a radio button is selected, the light is turned on, and only one light can be on at a time. No light is on when the program starts.
Experiment:12A)	

Source code:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class TrafficLight extends JFrame implements ActionListener {

    JRadioButton red, yellow, green;
    JPanel panel;

    public TrafficLight() {

        // Create radio buttons
        red = new JRadioButton("Red");
        yellow = new JRadioButton("Yellow");
        green = new JRadioButton("Green");

        // Group to allow only one selection
        ButtonGroup group = new ButtonGroup();
        group.add(red);
        group.add(yellow);
        group.add(green);

        // Panel to hold radio buttons
        JPanel radioPanel = new JPanel();
        radioPanel.add(red);
        radioPanel.add(yellow);
        radioPanel.add(green);

        // Panel where lights will be drawn
        panel = new JPanel() {
            public void paintComponent(Graphics g) {
```

```
super.paintComponent(g);

// Background
g.setColor(Color.darkGray);
g.fillRect(50, 30, 120, 260);

// Red Light
if (red.isSelected())
    g.setColor(Color.red);
else
    g.setColor(Color.gray);
g.fillOval(80, 40, 60, 60);

// Yellow Light
if (yellow.isSelected())
    g.setColor(Color.yellow);
else
    g.setColor(Color.gray);
g.fillOval(80, 120, 60, 60);

// Green Light
if (green.isSelected())
    g.setColor(Color.green);
else
    g.setColor(Color.gray);
g.fillOval(80, 200, 60, 60);
}
};

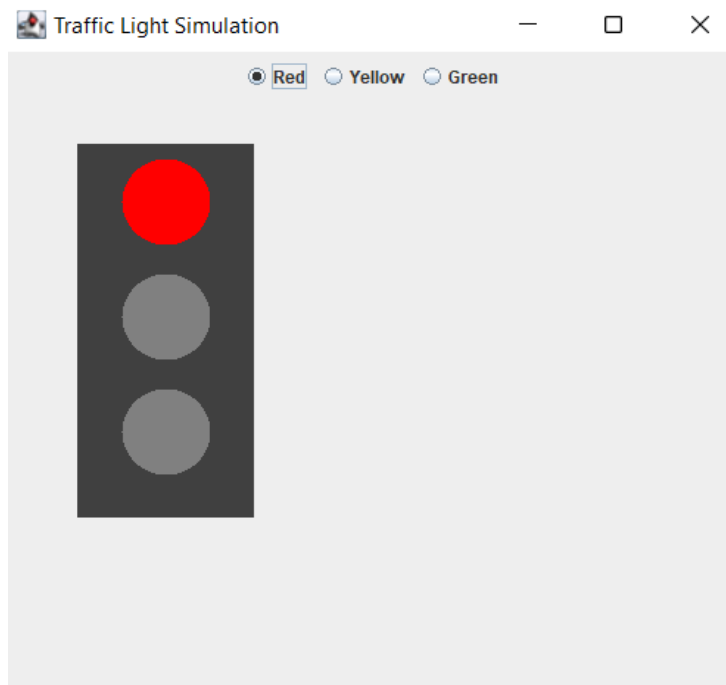
red.addActionListener(this);
yellow.addActionListener(this);
green.addActionListener(this);

add(radioPanel, BorderLayout.NORTH);
add(panel, BorderLayout.CENTER);

setTitle("Traffic Light Simulation");
setSize(250, 350);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```



```
}  
  
public void actionPerformed(ActionEvent e) {  
    panel.repaint(); // Refresh lights  
}  
  
public static void main(String[] args) {  
    new TrafficLight();  
}  
}
```

OUTPUT:

Date: 22/01/2026	Write a Java program that allows the user to draw lines, rectangles and ovals.
Experiment:12B)	

Source code:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ShapeDrawer extends JFrame implements ActionListener {

    JRadioButton lineBtn, rectBtn, ovalBtn;
    JPanel drawPanel;

    public ShapeDrawer() {

        // Radio buttons
        lineBtn = new JRadioButton("Line");
        rectBtn = new JRadioButton("Rectangle");
        ovalBtn = new JRadioButton("Oval");

        // Grouping radio buttons
        ButtonGroup group = new ButtonGroup();
        group.add(lineBtn);
        group.add(rectBtn);
        group.add(ovalBtn);

        JPanel topPanel = new JPanel();
        topPanel.add(lineBtn);
        topPanel.add(rectBtn);
        topPanel.add(ovalBtn);

        // Drawing panel
        drawPanel = new JPanel() {
            public void paintComponent(Graphics g) {
                super.paintComponent(g);
            }
        };
    }
}
```

```
// Draw based on selected shape
if (lineBtn.isSelected()) {
    g.drawLine(50, 50, 200, 200);
} else if (rectBtn.isSelected()) {
    g.drawRect(50, 50, 150, 100);
} else if (ovalBtn.isSelected()) {
    g.drawOval(50, 50, 150, 100);
}
}
};

// Add listeners
lineBtn.addActionListener(this);
rectBtn.addActionListener(this);
ovalBtn.addActionListener(this);

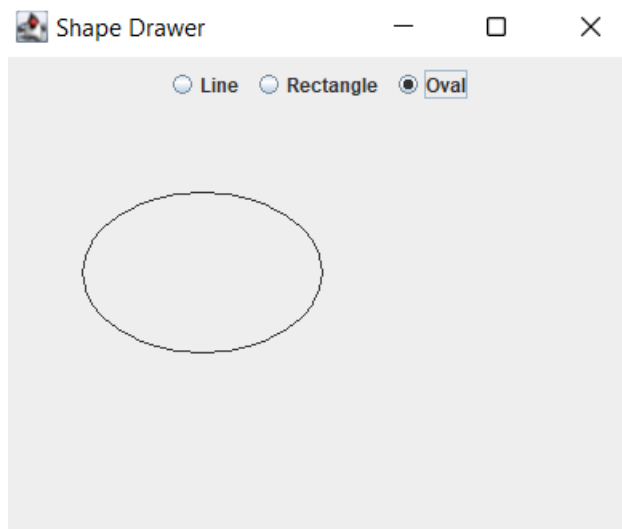
add(topPanel, BorderLayout.NORTH);
add(drawPanel, BorderLayout.CENTER);

setTitle("Shape Drawer");
setSize(300, 300);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public void actionPerformed(ActionEvent e) {
    drawPanel.repaint(); // Redraw shape
}

public static void main(String[] args) {
    new ShapeDrawer();
}
}
```

OUTPUT:



Date: 22/01/2026	Create a table in Table.txt file such that the first line in the file is the header, and the remaining lines correspond to rows in the table. The elements are separated by commas. Write a java program to display the table using J Table component
Experiment:12C)	

Source code:

```
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.util.*;

public class TableFromFile extends JFrame {

    public TableFromFile() {

        // Read file and convert to table format
        Vector<String> columnNames = new Vector<>();
        Vector<Vector<String>> data = new Vector<>();

        try {
            BufferedReader br = new BufferedReader(new
            FileReader("Table.txt"));
            String line;

            // Read header (first line)
            if ((line = br.readLine()) != null) {
                String[] header = line.split(",");
                for (String h : header) {
                    columnNames.add(h.trim());
                }
            }

            // Read rows
            while ((line = br.readLine()) != null) {
```

```
String[] row = line.split(",");
Vector<String> rowData = new Vector<>();

for (String element : row) {
    rowData.add(element.trim());
}
data.add(rowData);
}

br.close();
}
catch (Exception e) {
    e.printStackTrace();
}

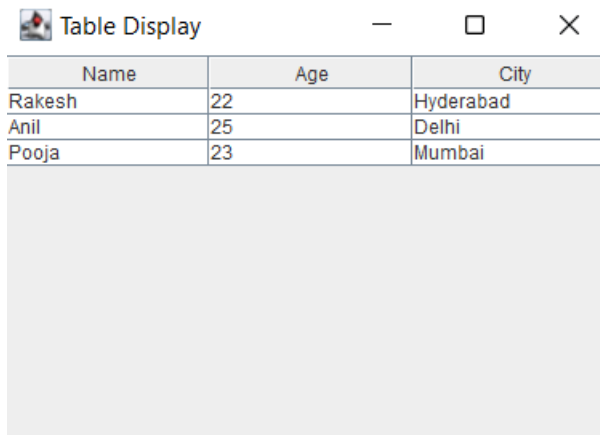
// Create JTable
JTable table = new JTable(data, columnNames);
JScrollPane sp = new JScrollPane(table);
add(sp);

setTitle("Table Display");
setSize(400, 300);
setVisible(true);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

public static void main(String[] args) {
    new TableFromFile();
}
}
```

Table.txt

```
Name, Age, City
Rakesh, 22, Hyderabad
Anil, 25, Delhi
Pooja, 23, Mumbai
```

OUTPUT:A screenshot of a Java Swing window titled "Table Display". The window contains a table with three columns: "Name", "Age", and "City". The table has three data rows: "Rakesh" (Age 22, City Hyderabad), "Anil" (Age 25, City Delhi), and "Pooja" (Age 23, City Mumbai). The window has standard OS controls (minimize, maximize, close) in the title bar.

Name	Age	City
Rakesh	22	Hyderabad
Anil	25	Delhi
Pooja	23	Mumbai